

REDES NEURAIS ARTIFICIAIS E MACHINE LEARNING

TEMA 1 – ARQUITETURA DE REDES NEURAIS ARTIFICIAIS

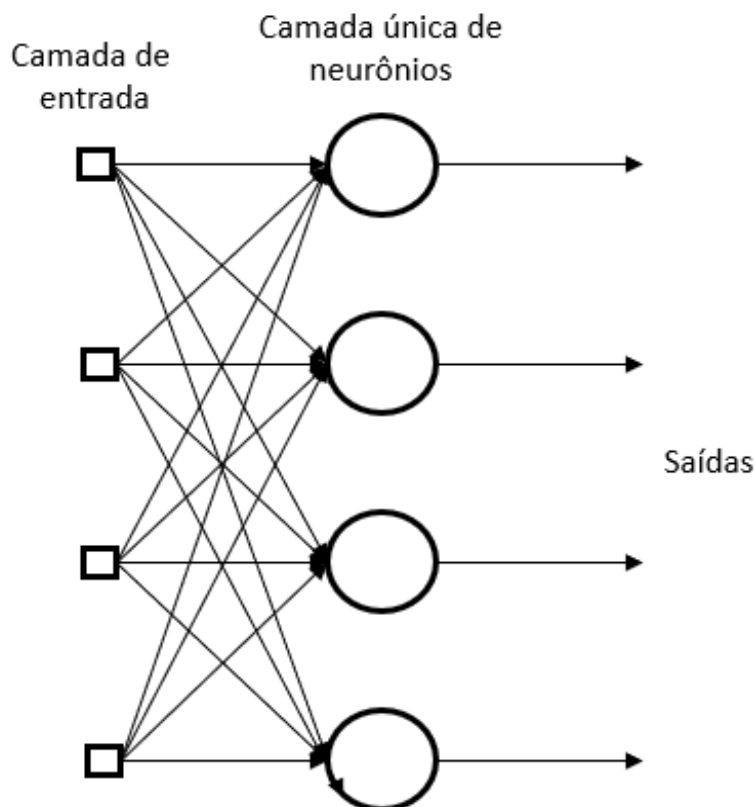
Como já estudado, um neurônio individual não consegue executar tarefas relacionadas a reconhecimento de padrões ou classificação. É necessário que mais neurônios estejam interconectados entre si, de forma a propagar os sinais de entrada e modular por um número maior de pesos sinápticos para poder aprender de forma efetiva. A arquitetura desempenha um papel essencial na forma com a rede processa as informações e também na forma com os algoritmos de aprendizagem trabalham. Na arquitetura de uma RNA, surge o conceito de *camada*. Pode-se dizer que uma camada contém um conjunto de neurônios que são acionados no mesmo estágio, ao mesmo tempo. O número de neurônios em uma camada dependerá principalmente do problema sendo modelado. De acordo com a arquitetura, Haykin (2001, p. 46-49) define três tipos de RNA: de *camada única*, de *múltiplas camadas* e *redes recorrentes*. A essa classificação é oportuno também adicionar as redes totalmente conectadas e redes parcialmente conectadas.

1.1 RNA de camada única

Consideradas como redes mais simples, as RNA de camada única contêm nas entradas e saídas dos seus neurônios as próprias entradas e saídas, respectivamente, da RNA. Ela tem a característica de ser uma *rede alimentada adiante* (do inglês *feedforward*), pois as informações sempre se propagam em um sentido só, das entradas para as saídas. Na Figura 1 pode ser visualizada uma representação de uma RNA de camada única. Deve-se notar que cada entrada está conectada a todos os neurônios. A camada de neurônios também pode ser considerada a própria camada de saída da RNA.

A RNA de camada única pode ser utilizado em contextos nos quais o mapeamento de entrada e saída se apresenta como uma função linear, onde não há muito ruído, permitindo-se obter uma função aproximada e assim uma generalização sobre os dados com boa precisão. A RNA de camada única também pode ser encontrada em análise de componentes principais, como camada de pré-processamento dos dados e extração de características, utilizando-se um algoritmo de aprendizagem hebbiana (Haykin, 2001, p. 452).

Figura 1 – Diagrama de uma RNA alimentada adiante de camada única



Fonte: Haykin, 2001, p. 47.

1.2 RNA de múltiplas camadas

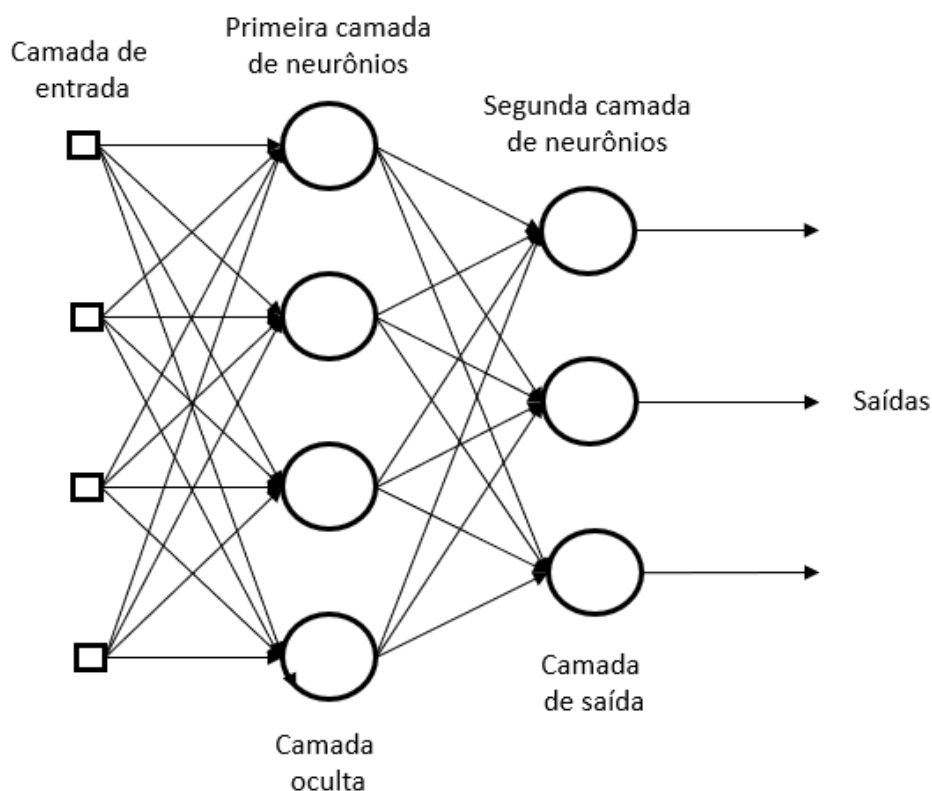
Em uma RNA que contenha múltiplas camadas existem as camadas *ocultas*, as quais podem ter suas entradas conectadas à camada de entrada da rede, ou ainda às saídas de outra camada oculta de neurônios (Haykin, 2001, p. 47). A ideia de adicionar camadas ocultas a uma RNA é a possibilidade de aumento dos pesos sinápticos, tornando a RNA mais propensa a generalizar sobre um contexto não linear de dados. Dessa forma, uma camada oculta possuirá um conjunto de saídas intermediárias que alimentarão outra camada à frente, até chegar à camada de saída. Na Figura 2, encontra-se uma RNA de duas camadas, uma camada oculta e outra camada de neurônios de saída.

Algoritmos de aprendizagem precisam ser adaptados para RNA com múltiplas camadas, em virtude da necessidade de retroalimentação para atualização dos pesos entre camadas ocultas, processo que é diferente com a camada de saída. RNA de múltiplas camadas são de uso mais geral do que as

RNA de camada única, e tanto a quantidade de camadas quanto a quantidade de neurônios nas camadas irão depender do problema de em questão.

Há uma relação de compromisso a ser alcançada entre a generalização de um modelo e a quantidade de camadas e neurônios: uma RNA com poucas camadas ocultas pode não ter pesos sinápticos suficientes para generalizar adequadamente sobre os dados, enquanto uma RNA com muitas camadas pode *congelar*, ou seja, aprender muito bem sobre os dados que fizeram parte do treinamento da RNA, mas ser altamente imprecisa com dados fora do conjunto de treinamento.

Figura 2 – Diagrama de uma RNA de múltiplas camadas



Fonte: Haykin, 2001, p. 48.

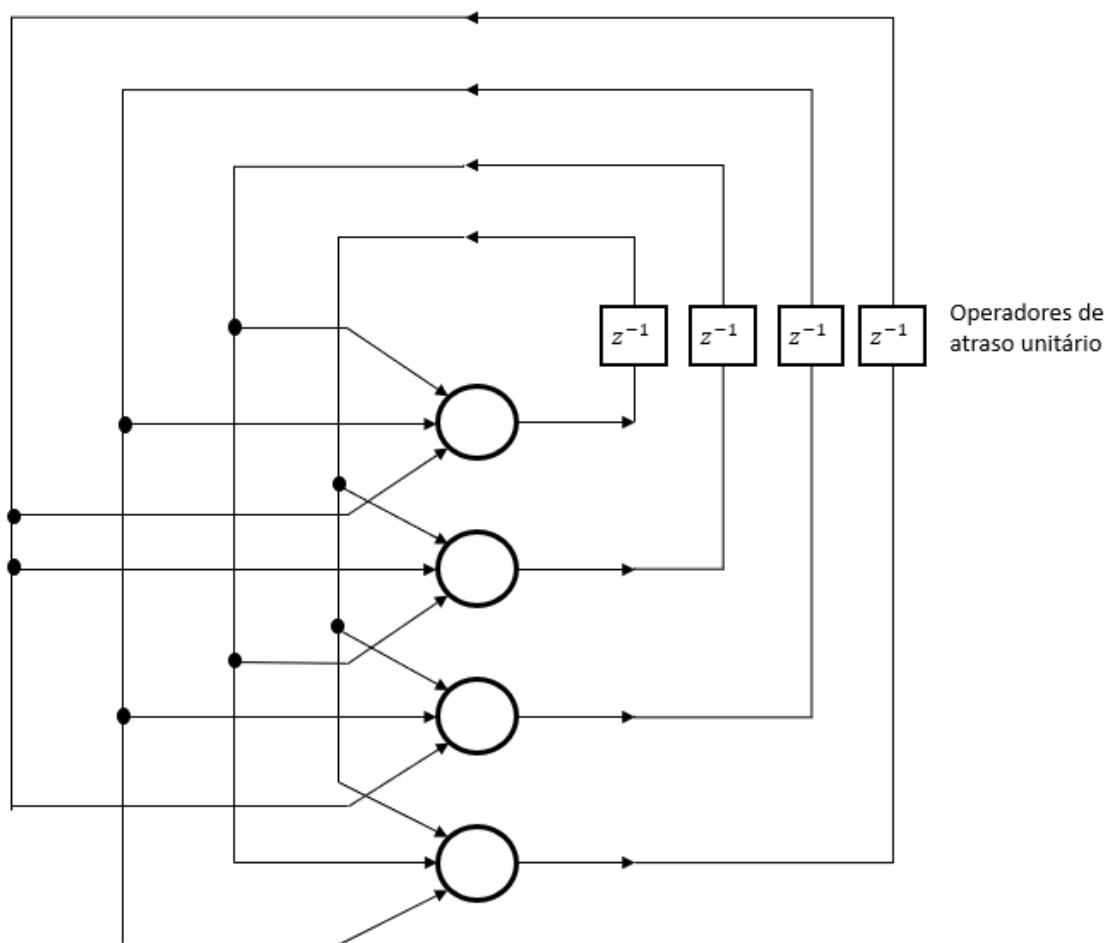
1.3 Redes recorrentes

As redes recorrentes são diferentes das RNA alimentada adiante, pois possuem pelo menos um laço de *feedback* ou *realimentação* na própria estrutura da RNA (Haykin, 2001, p. 48). Esse tipo de rede não deve ser confundido com a característica de certos algoritmos de aprendizagem, por exemplo, o LMS (*Least Mean Square*), que utilizam retroalimentação, e as redes onde são utilizados tais

algoritmos ainda são redes alimentadas adiante, sem recorrência na própria arquitetura.

A existência de laços de realimentação tem influência significativa sobre o desempenho e a aprendizagem da rede (Figura 3). As redes recorrentes possuem também os operadores de atraso unitário (simbolizado como z^{-1} , de acordo com a terminologia do estudo de processamento de sinais), para indicar que a saída em uma iteração t será retroalimentada na iteração $t + 1$ (e não na mesma iteração).

Figura 3 – Diagrama de uma rede recorrente



Fonte: Haykin, 2001, p. 48.

As redes recorrentes conseguem atender à necessidade de estabilização no treinamento de sistemas neurodinâmicos. Um exemplo de rede recorrente é a rede de Hopfield, cuja arquitetura se assemelha ao da Figura 3. Os padrões nessa rede são identificados como *atratores*, nos quais a rede de Hopfield busca obter o controle sobre onde esses atratores estão localizados no espaço de busca,

utilizando-se para o treinamento uma função de minimização de energia (Haykin, 2001, p. 732).

1.4 Redes totalmente conectadas

Nos exemplos das RNA de camada única e de múltiplas camadas, pode-se notar que, entre as camadas ocultas e de saída, todos os neurônios estavam conectados por pesos sinápticos. Dessa forma, as RNA totalmente conectadas exploram todas as possibilidades de conexão entre os neurônios das camadas próximas entre si.

Os algoritmos para aprendizagem das RNA totalmente conectadas consideram, para cada neurônio de uma camada, os pesos sinápticos com todos os outros neurônios de outras camadas, atualizando-os de acordo com a existência de correlação entre os padrões de entrada e saída para aquele par de neurônios. Se um peso sináptico atinge um valor alto, a influência dessa ligação é significativa e influencia a ativação nos neurônios adiante; se o peso sináptico é próximo de zero, então a conexão não é significativa e não contribui efetivamente para a ativação adiante.

De maneira geral, as redes totalmente conectadas são utilizadas para se executar de forma a cobrir da melhor forma possível o mapeamento de entrada e saída. Por exemplo, o uso de *perceptron* multicamada no estágio final de redes convolucionais geralmente são redes totalmente conectadas. Em termos de complexidade computacional, as RNA totalmente conectadas são as menos eficientes: por exemplo, em um algoritmo que seja necessária a tarefa de acessar os pesos sinápticos, numa camada oculta com m neurônios e uma camada de saída com n neurônios, a varredura dos pesos sinápticos exigirá um laço com $O(mn)$ operações.

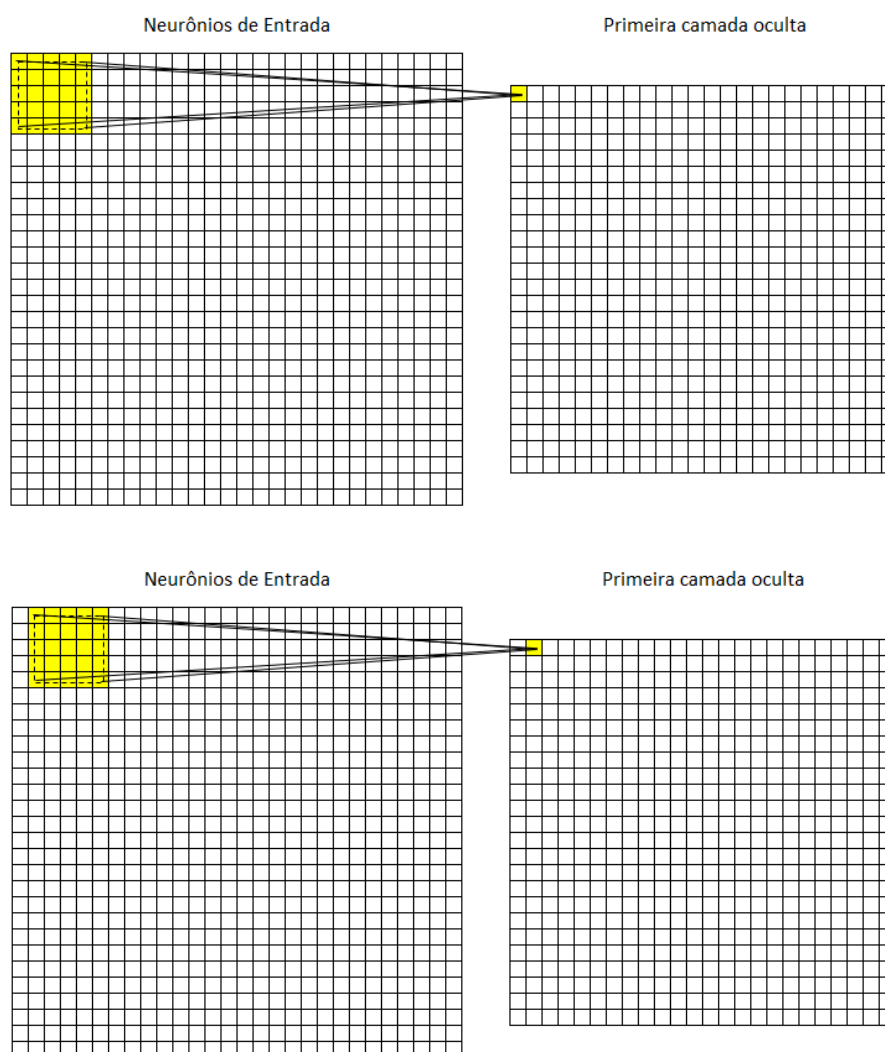
1.5 Redes parcialmente conectadas

As redes parcialmente conectadas se referem àquelas onde os neurônios de duas camadas próximas não estão totalmente conectados. O uso de conexões parciais entre neurônios de camadas diferentes pode se dar em função da existência de *informação prévia* durante o projeto da rede (Haykin, 2001, p. 54). Essa informação prévia se relaciona com o fato de que há um certo conhecimento prévio em como a rede precisa se comportar em relação ao ambiente. Dessa

forma, não seriam necessários todos os pesos sinápticos que podem existir entre duas camadas de neurônios.

Um exemplo é o caso da construção de uma rede que execute sua tarefa sobre *campos receptivos*. O uso de campos receptivos é comum em tarefas de reconhecimento de imagens, onde os conjuntos de neurônios ficam restritos a partes delimitadas do campo visual. Este é o caso das camadas convolucionais utilizadas em *deep learning*, que utilizam subamostragem e convolução, as quais utilizam campos receptivos reduzindo a necessidade de pesos sinápticos nessas camadas. Em alguns casos, não há necessidade inclusive de treinamento dos pesos dessas redes, assumindo valores fixos.

Figura 4 – Ilustração da conexão de pesos sinápticos entre duas camadas de neurônios, mostrando as conexões entre campos receptivos locais 5 x 5 conectados a somente um neurônio da camada seguinte.

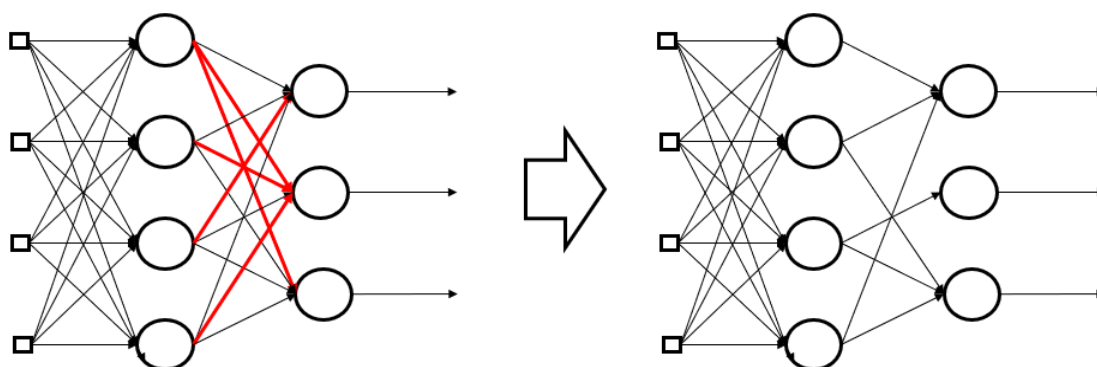


Em um segundo caso, a informação prévia pode direcionar a construção da rede, com base em uma *equação matemática* que informe quais conexões sinápticas deem ser colocadas. Por exemplo, no caso de reconstrução de imagens de tomografia, uma rede neural parcialmente conectada que implemente na arquitetura a expressão matemática por trás do processo de reconstrução de imagens em função de projeções de um feixe de radiação (transformada inversa de Radon). A RNA nesse caso não precisa ser treinada e utilizando-se dos dados alimentados na estrada, ela pode reconstruir a imagem interna do objeto na saída (Medeiros; Silva; Ribeiro, 2003).

O terceiro caso refere-se às redes em que, ao longo do processo de treinamento, o algoritmo utiliza uma *função de minimização com termo de regularização* (também chamado de termo de *punição de complexidade*), que fazem com que os pesos sinápticos assumam valores bem pequenos, que têm pouca influência no processo de ativação dos neurônios. Esse processo é denominado de *decaimento de pesos* (Haykin, 2001, p. 246). O decaimento pode melhorar a generalização de uma RNA, em virtude de retirar os pesos excessivos. No limite, esse processo origina a *eliminação de pesos*. À medida que os valores dos pesos tendem a zero, eles vão sendo eliminados, o que é denominado de *poda da rede*.

Dependendo da forma como o algoritmo de treinamento é construído, é possível reduzir as exigências de tempo e iterações, quanto à complexidade computacional. Portanto, enquanto nos casos da informação prévia por campos receptivos e por formalismo matemático, a arquitetura da rede é previamente definida. Nesse caso da função de minimização com regularização, a arquitetura é modificada dinamicamente, em tempo de treinamento da rede.

Figura 5 – Ilustração da poda de uma RNA com eliminação dos pesos próximos de zero (em vermelho, à esquerda)



TEMA 2 – PARADIGMAS DE APRENDIZAGEM DE REDES NEURAIS ARTIFICIAIS

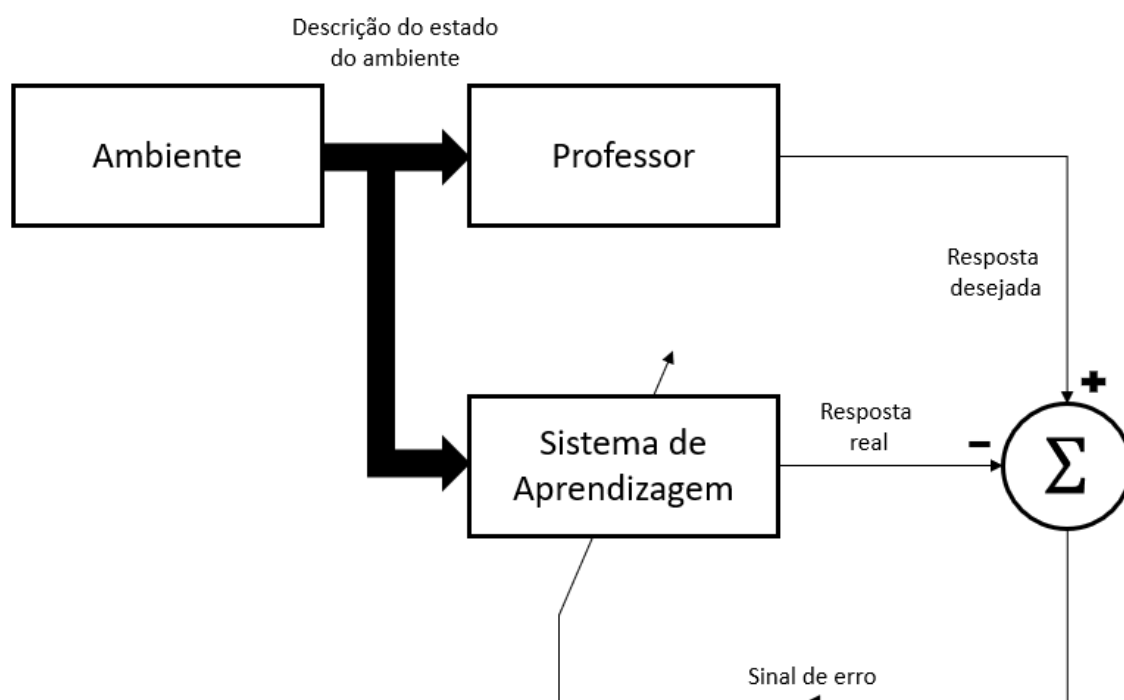
Os paradigmas de aprendizagem se referem à forma como os algoritmos de aprendizagem das RNA são executados, levando-se em consideração os dados disponíveis no ambiente para que a RNA possa então aprender alguma coisa sobre ele. A aprendizagem das RNA pode acontecer *com um professor*, *por reforço* e *sem professor*, ou ainda, por último, *sem professor*, também chamada de *não supervisionada*.

2.1 Aprendizagem com um professor

A aprendizagem com um professor é também denominada de *aprendizagem supervisionada*. Na Figura 6 está mostrado um diagrama em blocos em como esse processo se desenvolve. De maneira geral, a RNA não possui nenhum conhecimento sobre o ambiente, porém o professor tem esse conhecimento. O conhecimento que o professor possui está representado na forma de exemplos em como as entradas e saídas são mapeadas. Dessa forma, o conhecimento do professor é empírico. No instante da aprendizagem da RNA, algum vetor é extraído do ambiente e enviado a ela e ao professor, sendo que o professor sabe a resposta, ou pelo menos o caminho para obter a resposta. Ou seja, ele tem a *resposta desejada*. Entretanto, a RNA tem grande probabilidade de fornecer uma *resposta real* ao estímulo com erro. É necessário então identificar o sinal de erro, resultante da diferença da resposta desejada com a resposta real.

O sinal de erro ajusta as crenças da RNA (pesos sinápticos) de forma iterativa e gradativa, visando chegar ao mesmo *status* de conhecimento do professor. Ou seja, a RNA deverá ter a capacidade de *emular* o professor. Portanto, o conhecimento relativo ao ambiente que o professor possui é transferido gradativamente para a RNA. Quando uma condição de erro mínimo é alcançada, ou seja, praticamente não há diferença entre a resposta desejada e a resposta real, então a aprendizagem do ambiente é alcançada (Haykin, 2001, p. 88).

Figura 6 – Diagrama em blocos ilustrando a aprendizagem com um professor



Fonte: Haykin, 2001, p. 88.

Em resumo, a aproximação do conhecimento da RNA com o do professor é um processo de busca em um espaço multidimensional de estados em uma superfície de erro. O erro que é calculado refere-se a uma média dos erros de todos os exemplos, e a noção geral é a de encontrar um ponto onde essa superfície de erro assume um valor mínimo. Por isso é importante que o conhecimento do ambiente pelo professor seja o mais representativo possível: um conjunto de exemplos com viés não irá implicar um aprendizado adequado pela RNA, mesmo que ela atinja um valor mínimo de erro.

De maneira geral, em *machine learning*, um banco de dados com exemplos é dividido em dois conjuntos: um que ficará com o professor que irá *ensinar* efetivamente a RNA e outro com um professor que irá apenas *avaliar* a RNA – o

seu conhecimento não será usado para a aprendizagem da RNA. O primeiro conjunto é dito *conjunto de treinamento* e o segundo o *conjunto de teste*. Um processo de treinamento pode utilizar diferentes combinações aleatórias dos conjuntos, de acordo também com critérios de divisão do banco de dados e das quantidades de exemplos.

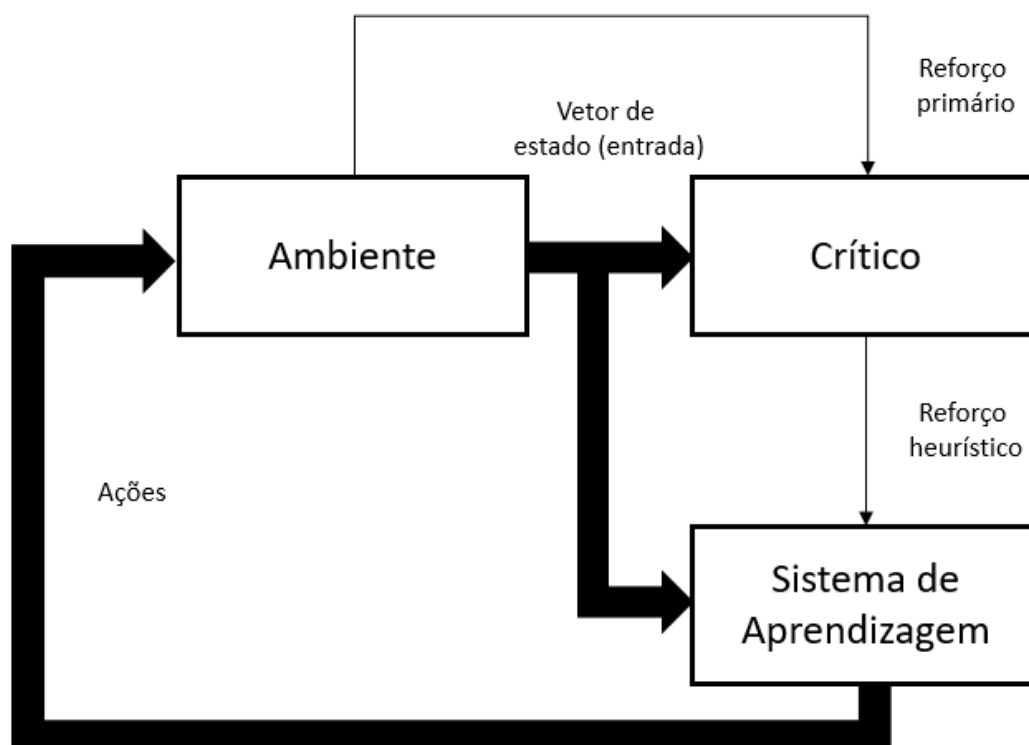
2.2 Aprendizagem por reforço sem um professor

Enquanto a aprendizagem supervisionada se refere a um problema cognitivo da RNA a ser resolvido com a ajuda de um professor, a aprendizagem por reforço é um problema de aprendizagem que se baseia em comportamento, ou seja, ela acontece mediante a interação do sistema de aprendizagem com o seu ambiente, buscando realizar um objetivo apesar de haver incertezas no ambiente. A aprendizagem por reforço se aplica a problemas relacionados com a programação dinâmica. Por exemplo, problemas de otimização logística.

Na aprendizagem por reforço, o mapeamento entrada-saída é obtido por meio da interação contínua com o ambiente. Da mesma forma que o aprendizado supervisionado, busca-se minimizar um índice de desempenho. A Figura 7 mostra o diagrama em blocos desse paradigma de aprendizagem, o qual é elaborado ao redor de um *crítico* que converte um sinal de reforço recebido do ambiente em um sinal mais qualificado, denominado de *reforço heurístico* (Haykin, 2001). O sistema aprende por reforço atrasado: ou seja, ele vai observando uma sequência temporal dos estímulos apresentados à RNA.

O objetivo é a minimização de uma função custo para avançar, definida como a expectativa do custo cumulativo de ações tomadas ao longo de uma sequência passos, em vez de simplesmente do curso imediato. Pode acontecer que certas ações tomadas anteriormente naquela sequência de passos de tempo sejam de fato os melhores determinantes do comportamento global do sistema. A função da máquina de aprendizagem, que constitui o segundo componente do sistema, é descobrir estas ações e realimentá-las para o ambiente. (Haykin, 2001, p. 89-90)

Figura 7 – Diagrama em blocos ilustrando a aprendizagem por reforço sem um professor



Fonte: Haykin, 2001, p. 90.

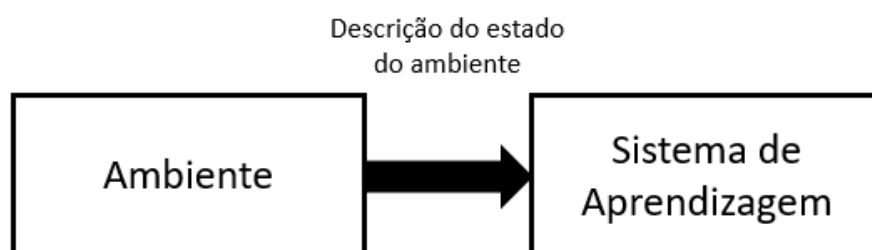
A aprendizagem por reforço sem um professor é difícil de ser realizada, pois justamente não há um conjunto de exemplos que venha a servir como professor, e também em como a RNA conseguirá discriminar se algo é um reforço positivo ou negativo (denominado de problema de atribuição de crédito), pois o reforço primário é apenas de avaliação do resultado. Entretanto, esse paradigma é muito atrativo, pois permite o desenvolvimento de sistemas que irão aprender com base na sua experiência de interação com o ambiente (Haykin, 2001, p. 90). A construção de agentes para jogos é um dos exemplos utilizados para esse paradigma de aprendizagem.

2.3 Aprendizagem não supervisionada

A aprendizagem não supervisionada também é denominada de *auto-organizada*. Aqui, não há um professor para direcionar o desempenho da rede. São dadas à RNA condições para que possa fazer uma medida de desempenho independente da tarefa, e a rede precisa alcançar um nível de representação dos

dados, havendo inclusive um conjunto de parâmetros para balizar o aprendizado (Haykin, 2001, p. 91). Quando a RNA alcança este nível, onde ela consegue fazer emergir as regularidades subjacentes nos dados, ela é capaz de atribuir classes aos dados organizados nesta representação interna obtida.

Figura 8 – Diagrama em blocos ilustrando a aprendizagem por reforço sem um professor



Fonte: Haykin, 2001, p. 91.

Nos sistemas de aprendizagem das RNA não supervisionadas, geralmente é utilizada a regra de aprendizagem competitiva. Uma camada competitiva de neurônios é ativada, de maneira a seguir uma regra do tipo “o vencedor leva tudo”, análoga ao potencial de ação do neurônio biológico. O escopo de alcance de um neurônio sobre os outros é modificado parametricamente, de forma que as atuações iniciais exerçam grandes alterações na representação, diminuindo gradativamente até chegar em um nível onde o neurônio consegue apenas ser ativado apenas em relação aos seus vizinhos mais próximos. Quando isso acontece, uma representação global tende a ser alcançada.

TEMA 3 – REGRAS DE APRENDIZAGEM DE REDES NEURAIS ARTIFICIAIS

A capacidade de aprendizagem de uma RNA é a característica que a diferencia de outras técnicas ou ferramentas de Inteligência Artificial. A RNA, além de aprender com o ambiente, pode melhorar também o seu desempenho ao longo de treinamentos posteriores. Para compreender melhor a forma como a aprendizagem acontece no âmbito das RNA, é necessário evidenciar o entendimento do que vem a ser aprendizagem dentro do contexto das redes neurais e separar de outros conceitos de áreas como psicologia e pedagogia.

Aprendizagem é um processo pelo qual os parâmetros livres de uma rede neural são adaptados através de um processo de estimulação pelo ambiente no qual a rede está inserida. O tipo de aprendizagem é determinado pela maneira pela qual a modificação dos parâmetros ocorre. (Haykin, 2001, p. 75)

Essa definição implica que:

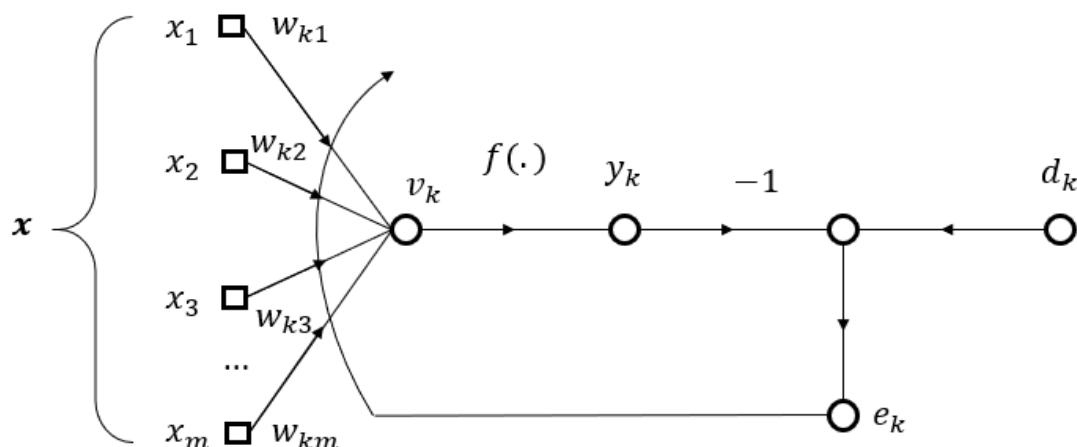
- a RNA é estimulada pelo ambiente;
- a RNA sofre modificações nos seus parâmetros livres; e
- a RNA responde de uma maneira distinta ao ambiente.

Dessa maneira, para resolver o problema de aprendizagem de uma RNA, é necessário definir um conjunto de regras bem estabelecidas para a solução desse problema. A esse conjunto denomina-se *algoritmo de aprendizagem*. Não existe um algoritmo único de aprendizagem de uma RNA: basicamente, o funcionamento do algoritmo depende da forma em como se farão as atualizações dos pesos sinápticos. Existem pelo menos cinco formas pelas quais uma RNA pode aprender por meio de um algoritmo: por correção de erro, com base em memória, pela regra de Hebb, por aprendizagem competitiva e aprendizagem estocástica de Boltzmann.

3.1 Aprendizagem por correção de erro

A aprendizagem por correção de erro foi uma das primeiras abordagens de modificação dos pesos sinápticos, inspirado no conceito de autorregulação e controle da cibernética. Conforme a Figura 9, um vetor de sinal $x = (x_1, x_2, \dots, x_m)$ é alimentado a um neurônio qualquer k e então modulado pelos pesos sinápticos w_{kj} , sendo somado na saída intermediária v_k . Pela aplicação da função de ativação obtém-se a saída real $y_k = f(v_k)$. Essa saída é comparada com a resposta desejada d_k , produzindo então o sinal de erro $e_k = d_k - y_k$. Este sinal de erro irá acionar um mecanismo de controle que irá modificar gradativamente os pesos sinápticos w_{kj} , em direção à resposta desejada.

Figura 9 – Grafo do fluxo do sinal por um neurônio, indicando a correção de erro



Fonte: Haykin, 2001, p. 77.

O objetivo em direção ao menor erro é alcançado por meio da minimização de uma função custo global $\mathcal{E} = \frac{1}{2} e_k^2$. Essa regra de aprendizagem aplicada para minimização do erro é denominada de *regra delta* ou *regra de Widrow-Hoff* (Haykin, 2001, p. 77). Essa regra requer o cálculo de um ajuste Δw_{kj} que irá modificar o peso sináptico w_{kj} calculado como

$$\Delta w_{kj} = \eta e_k x_j$$

O ajuste, também chamado de *delta*, é um valor que resulta da multiplicação do sinal de erro pelo sinal de entrada respectivo. A taxa de aprendizagem $\eta \in [0,1]$ indica que apenas uma pequena parte do sinal do erro pelo sinal de entrada é aproveitado para efetuar a mudança gradativa nos pesos sinápticos, mostrando que apenas uma parte do sinal do erro é retroalimentado. O valor do peso sináptico w_{kj} no instante $n + 1$, será o valor do instante n mais o ajuste calculado:

$$w_{kj}(n + 1) = w_{kj}(n) + \Delta w_{kj}(n)$$

A regra delta pode ser utilizada para o treinamento de um *perceptron* de camada única com o *algoritmo LMS* (mínimo quadrado médio), por exemplo, e serve de base para algoritmos mais elaborados como o *algoritmo da descida de gradiente*, bastante utilizado em *perceptrons* multicamada.

3.2 Aprendizagem baseada em memória

A aprendizagem baseada em memória se beneficia das experiências anteriores que estão armazenadas na forma de pares de entrada-saída, classificados de maneira correta $\{(x_i, d_i)\}, i = 1..N$, onde x_i representa um vetor de sinais de entrada e d_i representa a resposta desejada (correta) respectiva. Esse tipo de aprendizagem é comum em tarefas de classificação de padrões. Por exemplo, para classificar padrões em duas classes $\mathcal{C}_1$ e $\mathcal{C}_2$, d_i pode ser 0 para $\mathcal{C}_1$ e 1 para $\mathcal{C}_2$. Quando se apresenta um vetor x_t que não faz parte da memória de pares, um algoritmo de classificação deverá analisar os dados relativos aos vizinhos próximos a x_t . A maioria dos algoritmos difere quanto a dois ingredientes essenciais para o funcionamento:

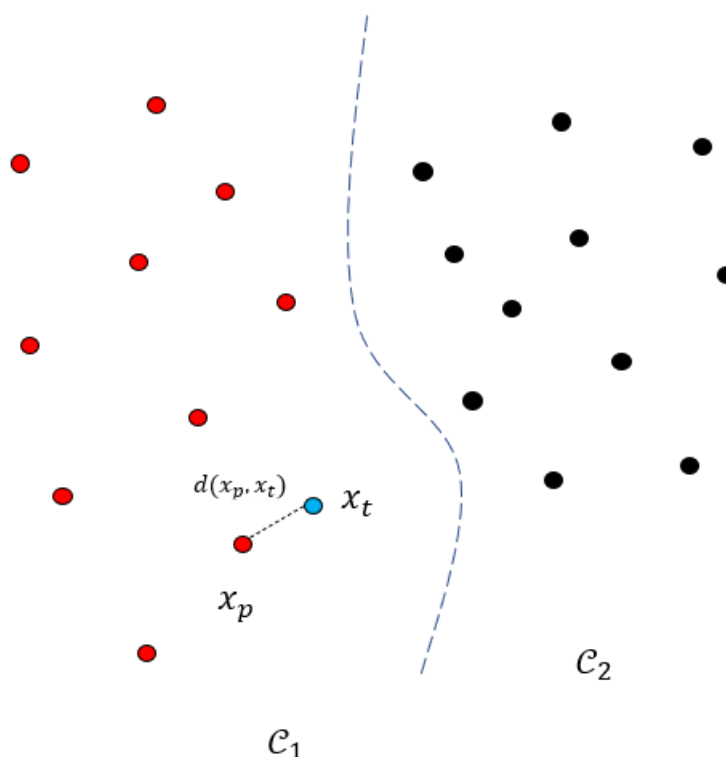
- a definição do critério de vizinhança local de x_t ; e
- a regra de aprendizagem aplicada aos exemplos de treinamento nesta vizinhança local de x_t .

Um tipo bem simples de aprendizagem baseada em memória é conhecido como *regra do vizinho mais próximo* (*nearest neighbor*). Aqui, a vizinhança é definida como os pares de treinamento mais próximo da vizinhança local de x_t . Diz-se que um vetor da memória x_p é o vizinho mais próximo de x_t se

$$d(x_p, x_t) = \min_i d(x_i, x_t)$$

Geralmente se associa $d(x_i, x_t)$ como a distância euclidiana entre os vetores de sinais de entrada x_i e x_t . Se atender a esse critério, então x_t será da mesma classe que x_p (Figura 10).

Figura 10 – Ilustração da regra de aprendizagem baseada em memória



Exemplos de algoritmos que trabalham com regras de aprendizagem baseada em memória são o *k-Means* e o DBSCAN.

3.3 Aprendizagem hebbiana

Essa regra de aprendizagem está ligada aos trabalhos do neuropsicólogo Donald Hebb, sendo a mais antiga dentre as outras. Em 1949, ele publica no seu livro *The Organization of Behavior* a citação que fundamenta a denominada *regra de Hebb*.

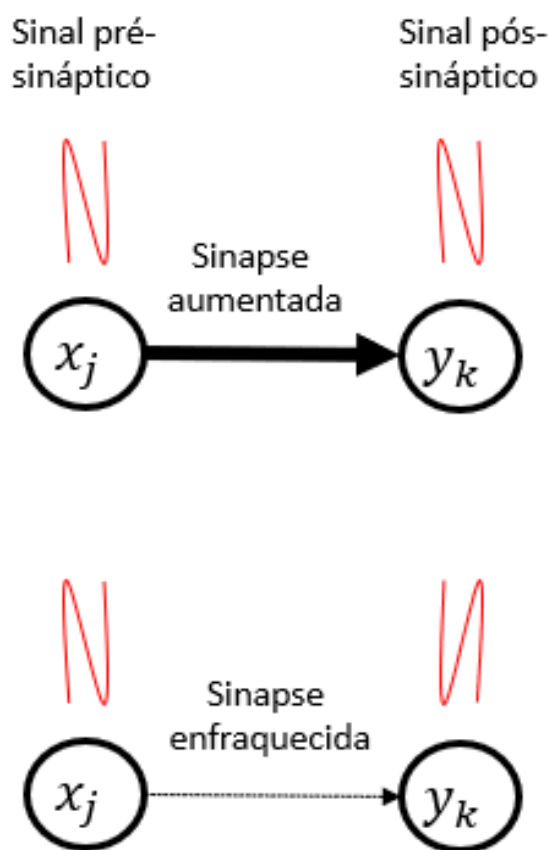
Quando um axônio da célula A está próximo o suficiente para excitar uma célula B e repetidamente ou persistentemente participa do disparo, algum processo de crescimento ou mudança metabólica ocorre em uma ou ambas as células, de modo que a eficiência de A, como uma das células disparando B, é aumentada. (Hebb, 1949, p. 62)

No contexto do paradigma de aprendizagem comportamental, a preocupação de Hebb era a da caracterização da aprendizagem associativa em nível celular. Ao longo do processo, isso iria resultar numa modificação permanente do padrão de atividade de um grupo de neurônios distribuído em uma região.

Trazendo a afirmação para o contexto das RNA, a regra poderia ser adaptada e reescrita em duas partes (Haykin, 2001, p. 80):

1. se dois neurônios em ambos os lados de uma sinapse são ativados simultaneamente, então a força daquela sinapse é seletivamente aumentada; e
2. se dois neurônios em ambos os lados de uma sinapse são ativados assincronamente, então aquela sinapse é seletivamente enfraquecida ou eliminada.

Figura 11 – Ilustração da regra de aprendizagem hebbiana



Essa seria, portanto, uma *sinapse hebbiana* (Figura 11), que usa um mecanismo que é:

- a. Dependente do tempo** – as modificações em uma sinapse hebbiana dependem do tempo exato da ocorrência dos sinais pré-sinápticos e pós-sinápticos;
- b. Local** – devido ao fato de que a sinapse é um transmissor local de informação entre os neurônios que a conectam;

- c. **Interativa** – as modificações dependem de uma interação entre as atividades pré-sináptica e pós-sináptica; e
- d. **Correlativa** – há uma conjunção entre as atividades pré-sináptica e pós-sináptica para a modificação da sinapse. Uma sinapse hebbiana é uma sinapse correlativa: “a correlação é de fato a base para o aprendizado” (Eggermont, 1990, citado por Haykin, 2001).

Um formalismo para tratar a regra de aprendizagem hebbiana considera um peso sináptico w_{kj} de um neurônio qualquer k que contenha respectivamente sinais pré-sinápticos x_j e pós-sinápticos y_k . O ajuste a ser feito no peso sináptico w_{kj} pode ser representado por:

$$\Delta w_{kj} = F(y_k, x_j)$$

onde $F(.)$ é uma função tanto do sinal pré-sináptico quanto do sinal pós-sináptico. A forma mais simples se pode ser descrita por:

$$\Delta w_{kj} = \eta y_k x_j$$

onde η é a taxa de aprendizagem. Esta equação mostra claramente a natureza correlativa de uma sinapse hebbiana, conhecida também como *regra do produto das atividades*. Algoritmos de treinamento que exploram a correlação entre variáveis, como a análise de componentes principais ACP, se enquadram nesta categoria. Ainda que a ACP seja uma técnica com derivação da estatística, ela pode ser calculada por uma RNA de camada única utilizando regra de aprendizagem hebbiana (Haykin, 2001, p. 452).

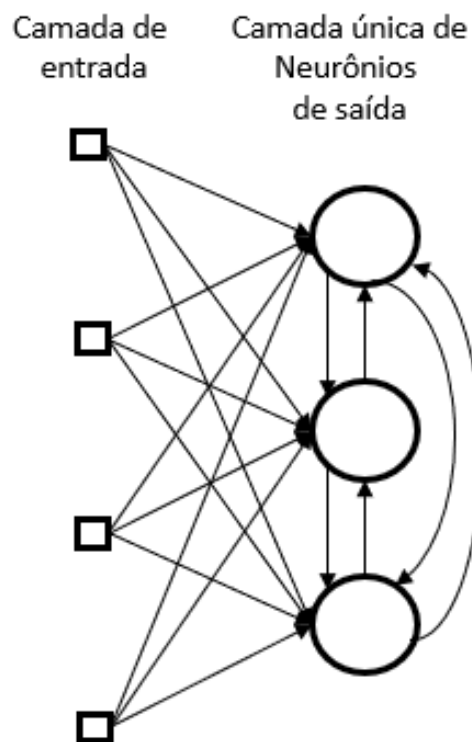
3.4 Aprendizagem competitiva

Nessa regra de aprendizagem, a característica principal é que os neurônios competem entre para efetuar o disparo, ou seja, tornarem-se ativos. Na *aprendizagem hebbiana*, é possível para vários neurônios estarem ativos ao mesmo tempo. Já na *aprendizagem competitiva*, apenas um neurônio está ativo em um certo instante. Essa característica torna a aprendizagem competitiva adequada para a descoberta de elementos capazes de melhorar a distinção entre padrões em uma tarefa de classificação (Haykin, 2001, p. 83).

Há três elementos fundamentais em uma regra de aprendizagem competitiva:

- um conjunto de neurônios que são todos iguais entre si, mas tendo pesos sinápticos distribuídos aleatoriamente, e que respondem de maneira diferenciada a uma estimulação de entrada;
- uma restrição imposta à “força” de um neurônio; e
- um mecanismo que permita ao neurônio disputar pelo direito de responder a um subconjunto de entradas de forma que apenas um neurônio de saída esteja ativado.

Figura 12 – Diagrama de uma rede de aprendizagem competitiva simples com conexões de alimentação adiante excitatórias e conexões laterais inibitórias



Fonte: Haykin, 2001, p. 84.

Em um modelo bem simplificado de aprendizagem competitiva, uma RNA possui uma única camada de neurônios de saída, com todos os neurônios conectados aos nós de entrada. A rede pode incluir conexões sinápticas de realimentação adicionais entre os neurônios que executam uma inibição lateral, onde cada neurônio tende a inibir o seu vizinho. Já as conexões de alimentação da camada de saída são todas excitatórias (Figura 12).

Um neurônio k será considerado vencedor se o valor do somatório dos estímulos de entrada v_k , também chamado de *campo local induzido*, tiver um valor maior do que todos os outros neurônios. Dessa forma, o sinal da resposta de saída y_k é assumido como o valor 1 (um), e o sinal relativo aos outros neurônios que perderam a competição será assinalado como 0 (zero).

$$y_k = f(x) = \begin{cases} 1, & v_k > v_j, \\ 0, & v_k \leq v_j \end{cases} \quad \forall j, j \neq k$$

O neurônio vai aprendendo ao longo do tempo a deslocar os pesos sinápticos das entradas inativas para as entradas ativas. Caso um neurônio não responda a um certo padrão de entrada, então não ocorrerá aprendizagem nesse neurônio. Caso um neurônio vença a competição, cada entrada destina uma certa proporção do seu peso sináptico e este peso será distribuído entre as entradas ativas. Em uma regra de aprendizagem competitiva, a variação Δw_{kj} aplicada ao peso sináptico é calculada da seguinte forma:

$$\Delta w_{kj} = \begin{cases} \eta(x_j - w_{kj}), & \text{caso o neurônio } k \text{ vença a competição} \\ 0, & \text{caso o neurônio } k \text{ perca a competição} \end{cases}$$

Os mapas auto-organizáveis (SOM – *self-organizing maps*), que transformam um padrão de dimensão arbitrária em um mapa discreto uni ou bidimensional de forma adaptativa e ordenado topologicamente (Haykin, 2001, p. 486), utilizam a regra de aprendizagem competitiva.

3.5 Aprendizagem de Boltzmann

A regra de aprendizagem de Boltzmann está associada a um algoritmo de aprendizagem estocástico inspirado na mecânica estatística. Ela leva esse nome em referência a Ludwig Boltzmann, cujas contribuições ajudaram a construir este campo no estudo da física. A mecânica estatística utiliza a teoria das probabilidades para estudar o comportamento de sistemas macroscópicos constituídos por um grande número de entidades microscópicas. Uma RNA projetada para funcionar com base no algoritmo de Boltzmann é denominada de *máquina de Boltzmann* (Haykin, 2001, p. 86).

Numa máquina de Boltzmann, a arquitetura é de uma rede recorrente: os neurônios atuam de forma binária, representando um estado ligado (+1) ou desligado (−1). Existe uma função de energia cujo valor é determinado pelos

diversos estados particulares dos neurônios que constituem a máquina de Boltzmann. A função de energia é expressa como:

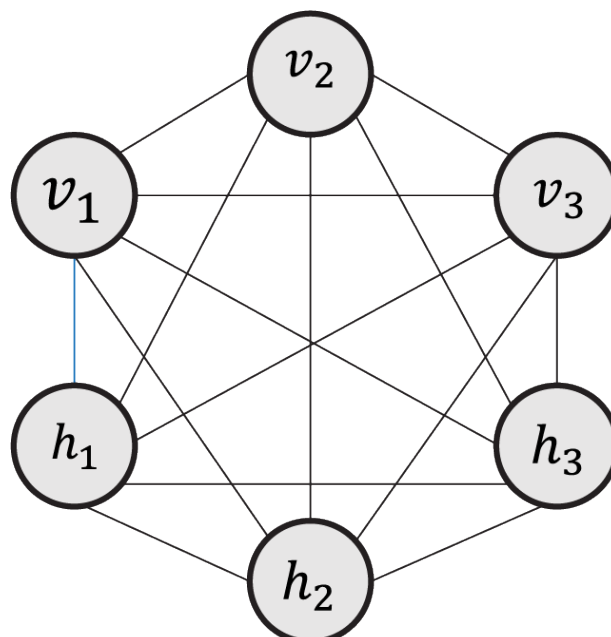
$$E = -\frac{1}{2} \sum_j \sum_{k, j \neq k} w_{kj} x_k x_j$$

onde w_{kj} é o peso sináptico que conecta o neurônio x_k ao neurônio x_j . A máquina opera escolhendo neurônios ao acaso (por exemplo, x_k) em certo passo do processo, e troca o estado do neurônio de x_k por $-x_k$, de acordo com uma probabilidade calculada a partir da equação:

$$P(x_k \rightarrow -x_k) = \frac{1}{1 + \exp(-\Delta E_k/T)}$$

Aqui, ΔE_k é a variação de energia resultante da troca $x_k \rightarrow -x_k$ (é preciso recalcular a função de energia com o valor alterado). A variável T é uma *pseudotemperatura*: a máquina de Boltzmann começa “quente” e vai resfriando ao longo do processo de aprendizagem, ou seja, o valor de T vai decaindo gradativamente. Ao final do processo, ela chega a uma pseudotemperatura na qual as probabilidades de transição $P(x_k \rightarrow -x_k)$ são muito pequenas, onde se pode dizer que a máquina de Boltzmann aprendeu, ou seja, quando a máquina atingiu o *equilíbrio térmico*.

Figura 13 – Diagrama de uma máquina de Boltzmann, mostrando os neurônios visíveis (v_i) e os neurônios ocultos (h_i)



Em uma máquina de Boltzmann, podem-se encontrar dois tipos de neurônios: os *visíveis* e os *ocultos*. Os neurônios visíveis estão ligados às entradas da rede, ou seja, assumem os valores +1 e -1 conforme as entradas. Já os neurônios ocultos sempre operam de forma livre (Figura 13). Com base nesta caracterização, a máquina de Boltzmann assume duas condições na operação:

- a. condição presa (*clamped*)** – os neurônios visíveis estão todos presos a estados específicos que são fixados pelo ambiente; e
- b. condição de operação livre (*free run*)** – todos os neurônios, visíveis e ocultos, podem operar livremente.

Finalmente, para calcular o ajuste dos pesos sinápticos, com base nas condições presa e de operação livre, define-se a correlação entre os estados dos neurônios j e k como ρ_{kj} . Adotam-se, portanto, dois tipos de correlação:

- i. a correlação positiva como ρ_{kj}^+ , que representa a correlação com a máquina na condição presa; e
- ii. a correlação negativa ρ_{kj}^- , com a máquina na sua condição livre.

Essas correlações são calculadas como a média sobre todos os estados possíveis que a máquina de Boltzmann pode assumir, quando está no equilíbrio térmico.

Conforme a regra de aprendizagem de Boltzmann, a variação Δw_{kj} é definida como sendo:

$$\Delta w_{kj} = \eta(\rho_{kj}^+ - \rho_{kj}^-), \quad j \neq k$$

Da mesma forma que as regras anteriores, η é a taxa de aprendizagem, e as correlações ρ_{kj}^+ e ρ_{kj}^- assumirão valores entre -1 e +1.

Máquinas de Boltzmann são utilizadas regularmente em estratégias de *deep learning*, sendo que uma variação, a máquina de Boltzmann restrita (RBM), simplifica o uso devido ao fato de não ter na arquitetura conexões entre neurônios do mesmo tipo. As RBM são as mais utilizadas, além de permitir valores reais para os neurônios ao invés de valores binários apenas (Coiurville; Bergstra; Bengio, 2011). O modelo tende a melhorar quanto mais camadas são adicionadas. RBM são utilizadas, por exemplo, em reconhecimento de fala.

TEMA 4 – TAREFAS DE APRENDIZAGEM DE REDES NEURAIS ARTIFICIAIS

Com base nos tipos de aprendizagem pelos quais uma RNA pode aprender algo sobre um conjunto de dados do ambiente, quais são os tipos de tarefas ou atividades que podem ser executadas por uma RNA? O cérebro humano, por meio de suas redes de neurônios, é capaz de executar com alto desempenho várias tarefas. A ideia geral é que uma RNA irá atuar em uma tarefa que é feita de algum modo por uma rede neural biológica. Por exemplo, processar um cenário com objetos e poder distinguir tais objetos por meio do processamento visual, ou mesmo fazer uma inferência com base em dados disponíveis no ambiente.

4.1 Associação de padrões

A associação é uma das tarefas extensamente executadas pela memória humana. Por exemplo, a memória mnemônica associa palavras a imagens, de maneira que, se uma imagem é apresentada posteriormente, somos capazes de recuperar a palavra associada a imagens semelhantes que foram gravadas na memória.

A memória associativa pode ser utilizada em uma RNA de duas formas (Haykin, 2001, p. 91):

- a. por **autoassociação**, onde uma RNA deve armazenar um conjunto de padrões apresentados repetidamente a ela; e
- b. por **heteroassociação**, onde um conjunto de padrões deve ser associado a outro conjunto de padrões.

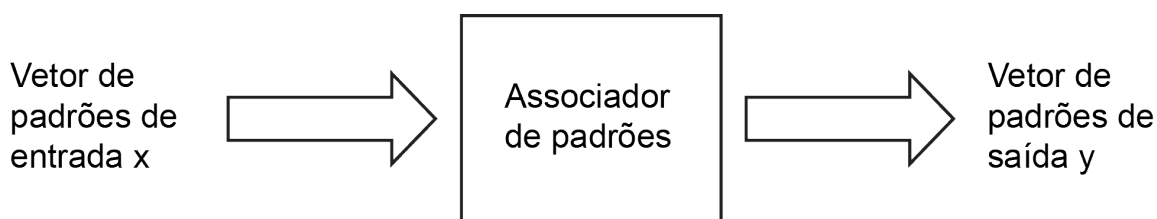
Na autoassociação, os padrões podem ser apresentados com nível de ruído nas entradas e a RNA deve recordar um padrão em particular, envolvendo alguma espécie de aprendizagem não supervisionada. Já a heteroassociação requer uma aprendizagem com professor ou supervisionada.

Se um padrão x_k é um padrão aplicado a uma memória associativa e y_k é um padrão memorizado, a associação de padrões será uma descrição dada por:

$$x_k \rightarrow y_k, \quad k = 1, 2, \dots, q.$$

onde q é o número de padrões armazenados na RNA. Dessa forma, o padrão x_k tem o papel de um estímulo apresentado à RNA capaz de recuperar o padrão armazenado y_k . No caso de uma memória associativa, $x_k = y_k$, enquanto que numa memória heteroassociativa, $x_k \neq y_k$.

Figura 14 – Tarefa de associação de padrões



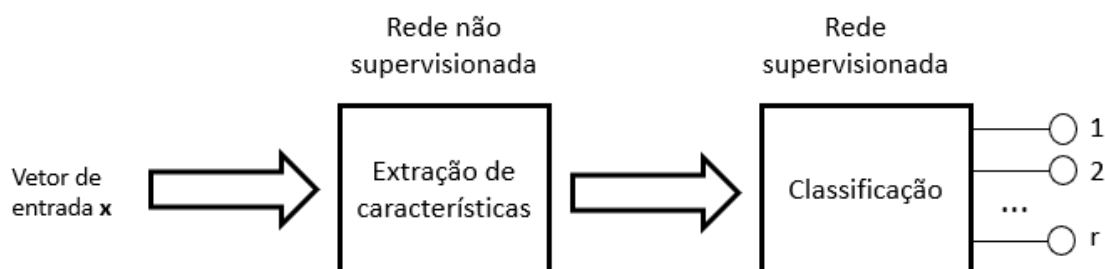
Fonte: Haykin, 2001, p. 92.

4.2 Reconhecimento de padrões

O reconhecimento de padrões é a atividade na qual um determinado padrão apresentado como um sinal de entrada é categorizado, ou seja, é atribuído a uma classe ou categoria dentre um número finito de classes. O reconhecimento de padrão é feito com uma rede supervisionada, onde são apresentados um conjunto de exemplos que são pré-classificados, fazendo com que a RNA aprenda a classificar adequadamente este conjunto pré-classificado. Após a fase de treinamento da RNA, na fase de teste é apresentado um outro conjunto de padrões que não fizeram parte da fase de treinamento, para verificar a acurácia da rede em classificar estes novos dados (Haykin, 2001, p. 92).

Uma RNA para reconhecimento de padrões é geralmente dividida em duas partes: a primeira é responsável pela extração de características, geralmente um processo não supervisionado, visando reduzir o tamanho do espaço de entrada (a quantidade de dimensões que possui o sinal do padrão a ser classificado), processo denominado de *redução de dimensionalidade*. Depois, vem uma rede supervisionada alimentada adiante (*feed forward*), que mapeia o padrão processado na primeira parte com a classe respectiva (Figura 15).

Figura 15 – Tarefa de reconhecimento de padrões



Fonte: Haykin, 2001, p. 93.

4.3 Aproximação de funções

Uma RNA pode ser utilizada também em tarefas de aproximação de funções. Supondo-se que exista num ambiente de uma RNA uma relação que é desconhecida e que permite gerar um vetor $\mathbf{d}$, a partir de um vetor de entrada $\mathbf{x}$, dada por:

$$\mathbf{d} = f(\mathbf{x})$$

A RNA pode ser utilizada para encontrar uma função $\mathcal{F}$ que aproxima $f(\cdot)$, relacionado a pares de padrões e saídas que são utilizados para se efetuar o treinamento, de forma que a função aproximada é dada por:

$$\mathcal{F} = \{(\mathbf{x}_i, \mathbf{d}_i)\}_{i=1}^N$$

Dessa forma, a RNA tem o objetivo de encontrar a função $\mathcal{F}$ que se aproxime o melhor possível de $f(\cdot)$, de forma que um erro ε seja o menor possível:

$$\|\mathcal{F}(\mathbf{x}) - f(\mathbf{x})\| < \varepsilon, \text{ para todo } \mathbf{x}.$$

A aproximação de funções pode ser feita de duas formas (Haykin, 2001, p. 94), como se segue.

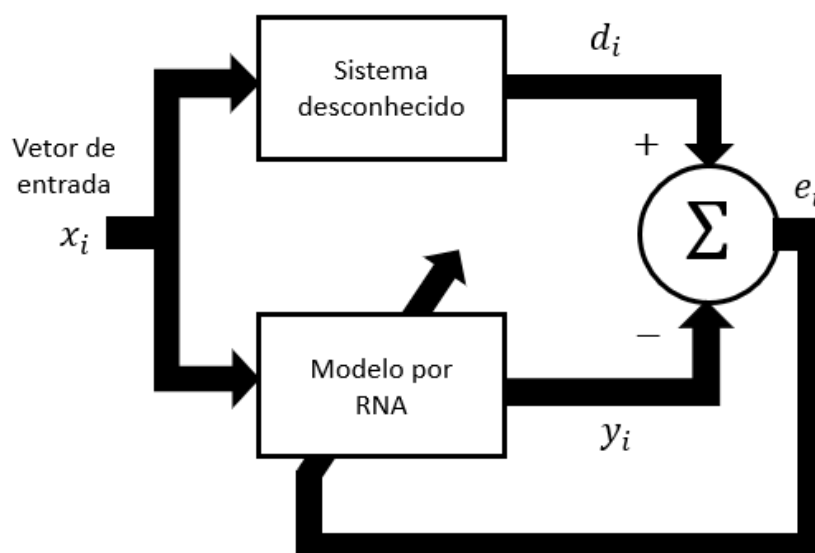
1. **Identificação de sistema:** o treinamento da RNA permite que se obtenha um modelo que emula um sistema desconhecido. À medida que as entradas são apresentadas, as saídas do modelo y_i são confrontadas com as saídas do sistema desconhecido d_i (através de diferença quadrática, evitando variações de sinal), de maneira a gerar o sinal de erro e_i . O sinal de erro é realimentado nos parâmetros livres do modelo (os pesos sinápticos) que faz o ajuste necessário em direção ao valor de erro mínimo. O sistema desconhecido deve ser sem memória, no sentido de que deve ser invariante no tempo (Figura 16).
2. **Sistema inverso:** da mesma forma, um sistema sem memória dado por $f(\cdot)$ fornece as saídas d_i a partir das entradas x_i , que são utilizadas para modelar um sistema inverso $f^{-1}(\cdot)$, de forma que pode se obter o sinal de entrada x_i a partir das saídas desejadas d_i :

$$\mathbf{x} = f^{-1}(\mathbf{d})$$

A saída aproximada y_i do modelo é confrontada, neste caso, com o próprio sinal de entrada x_i , o qual é o sinal que o modelo tenta emular. O erro e_i calculado

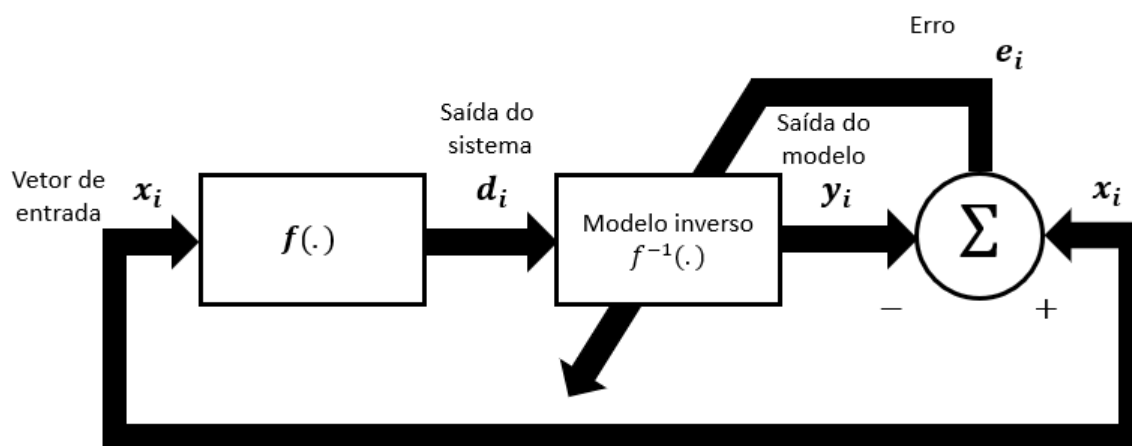
por diferença quadrática e realimentado novamente no modelo inverso (Figura 17).

Figura 16 – Tarefa de identificação de sistema em aproximação de função



Fonte: Haykin, 2001, p. 95.

Figura 17 – Tarefa de cálculo de sistema inverso em aproximação de função



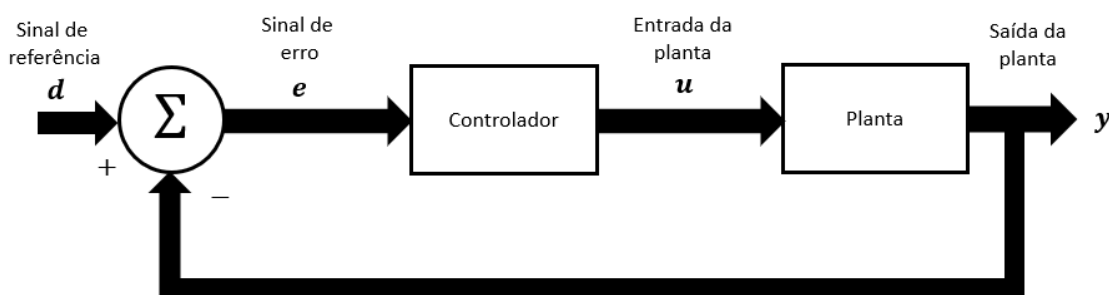
Fonte: Haykin, 2001, p. 95.

4.4 Controle

Uma RNA pode ser utilizada também para o controle de uma *planta*, a qual representa um processo de fabricação que tem seus parâmetros mantidos em condição controlada (por exemplo, processos químicos em uma refinaria). A RNA

é treinada de maneira que forneça as saídas na forma de ações que reajustam os parâmetros. Um sistema de controle envolve utilizar o sinal de saída y para calcular a realimentação para a entrada do sistema (na forma de derivativas parciais $\partial y / \partial u$), gerando-se um erro e que reajusta os parâmetros do controlador. Desta forma, os dados u são fornecidos novamente para a planta (Figura 18).

Figura 18 – Tarefa de cálculo de sistema inverso em aproximação de função



Fonte: Haykin, 2001, p. 96.

Uma RNA para controle de planta pode demandar uma *aprendizagem indireta*, que utiliza as medidas reais de entrada e saída da planta, gerando um modelo inicial que calcula a realimentação e reproduz uma cópia da planta; ou uma *aprendizagem direta*, que extrai os sinais de realimentação diretamente da planta e ajusta os parâmetros livres de maneira dinâmica (Haykin, 2001, p. 96).

4.5 Filtragem

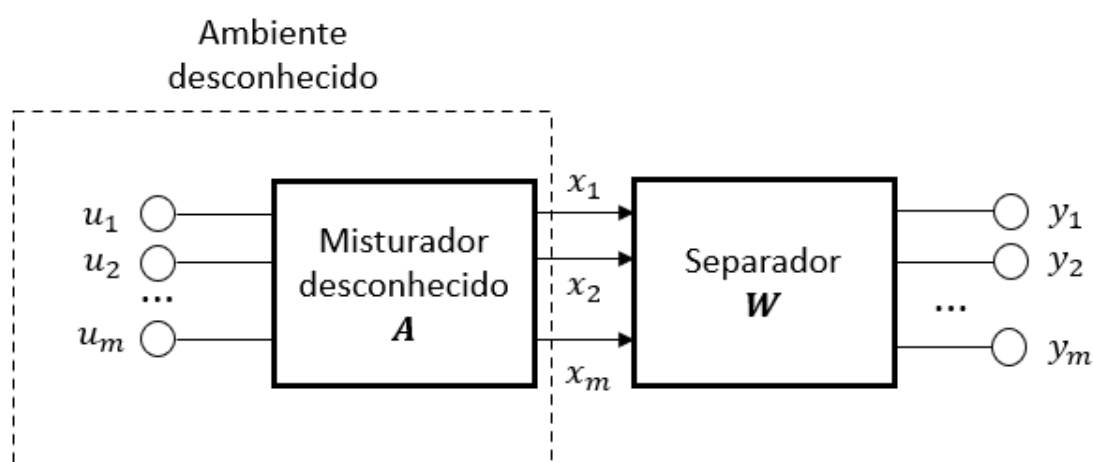
Uma RNA também pode ser utilizada para fazer a filtragem de sinais ruidosos. Na vida real, o uso de sensores pode introduzir ruído em algum sinal de entrada, o que pode induzir a interpretações errôneas do comportamento de um sistema. Outro problema é a transmissão deste sinal através de canais de informação que podem também ser ruidosos (Haykin, 2001, p. 97). Uma RNA pode ser utilizada então para três tipos de tarefas:

1. **filtragem** – extração de informação sobre um conjunto de sinais em um tempo discreto qualquer;
2. **suavização** – os dados medidos de um sinal são processados de forma a obter um componente de interesse dentro do sinal original, e que pode ser disponibilizado com atraso; e

3. previsão – o sinal medido é utilizado para prever quais serão os próximos valores a serem fornecidos no sinal de entrada.

Uma das tarefas em filtragem com RNA é o que se denomina de *separação cega de sinal* (Haykin, 2001, p. 97). Na Figura 19 está ilustrado como esse processo acontece. Em um ambiente desconhecido, só se conhece o sinal de entrada x_i que pode conter ruído de algum tipo, o qual modifica o sinal real u_i , porém desconhecido. O elemento separador lida com a informação fornecida por x_i de maneira a obter um sinal de saída y_i , que é uma tentativa de se conhecer o sinal original no ambiente u_i . Para esse separador, geralmente utiliza-se uma rede não supervisionada.

Figura 19 – Tarefa de filtragem (separação cega de fonte)



Fonte: Haykin, 2001, p. 96.

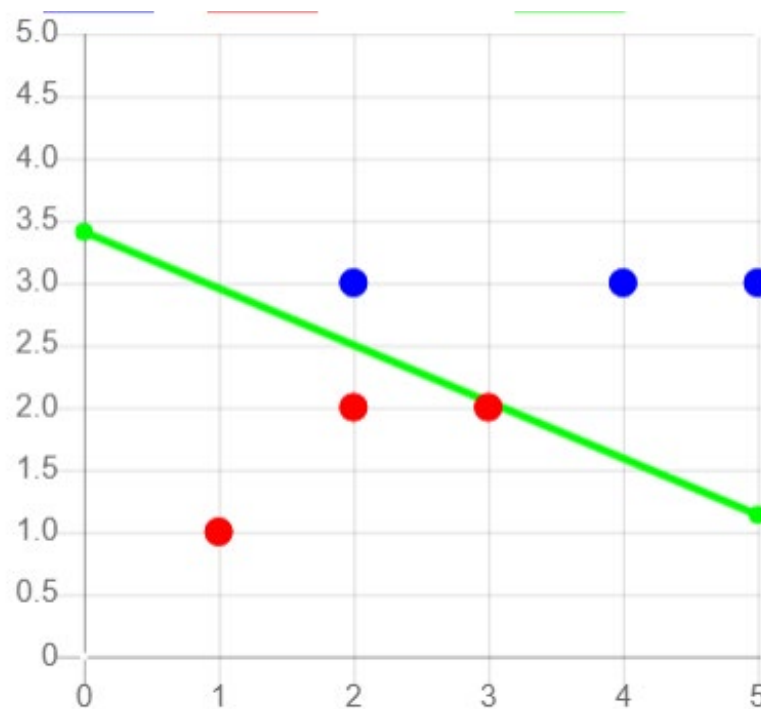
TEMA 5 – EXEMPLOS DE REDES NEURAIS ARTIFICIAIS

Pelo desenvolvimento dos temas anteriores, pode-se constatar que as RNA se diferenciam por vários aspectos: *arquitetura*, *paradigma*, *regras* e *tarefas de aprendizagem*. É oportuno, a partir daqui, conhecer então alguns tipos de implementações de RNA que podem executar as tarefas que foram mencionadas anteriormente.

5.1 Perceptrons

O *perceptron* é uma das RNA mais antigas. Proposto por Frank Rosenblatt em 1958, é considerado a forma mais simples para a tarefa de classificação de padrões que tem a característica de serem linearmente separáveis (padrões dispostos em lados opostos de um hiperplano – ver figura 20). O *perceptron* consiste basicamente em somente um neurônio com pesos sinápticos que podem ser ajustados e também uma entrada de bias. Construído dessa forma, o *perceptron* está limitado a fazer uma classificação contendo apenas duas classes (Haykin, 2001, p. 143). Um *perceptron* pode ser treinado utilizando-se o algoritmo do mínimo quadrado médio (LMS). O *perceptron* simples tem como função de ativação a função degrau (*step*).

Figura 20 – Em um *perceptron* simples com duas entradas (dimensões), o hiperplano que separa as classes é uma reta



Em função da limitação de um *perceptron* simples, um número maior de neurônios podem ser adicionados a uma RNA ou mais camadas ocultas de neurônios que se interligam entre si, permitindo aumentar o número de pesos sinápticos e a possibilidade de classificação e reconhecimento de padrões em um número de classes bem maior que duas opções apenas (Medeiros, 2007). Esse tipo de RNA é o *perceptron multicamada* (*Multilayer Perceptron – MLP*), uma opção bastante versátil para efetuar o processo de classificação, e que pode ser

treinada por vários tipos de algoritmos, dentre eles o de *descida de gradiente* (*gradient descent*). O *perceptron* multicamada permite o uso de funções de ativação com maior suavização, como a função sigmoide e a função tangente hiperbólica (Medeiros, 2018). Nos processos de *machine learning* ou *deep learning*, uma rede MLP geralmente é a última camada que se ocupa do mapeamento dos padrões obtidos com as classes do problema. Uma rede MLP pode ser utilizada para reconhecimento de padrões, regressão linear, em autocodificadores, correção e filtragem, dentre outros.

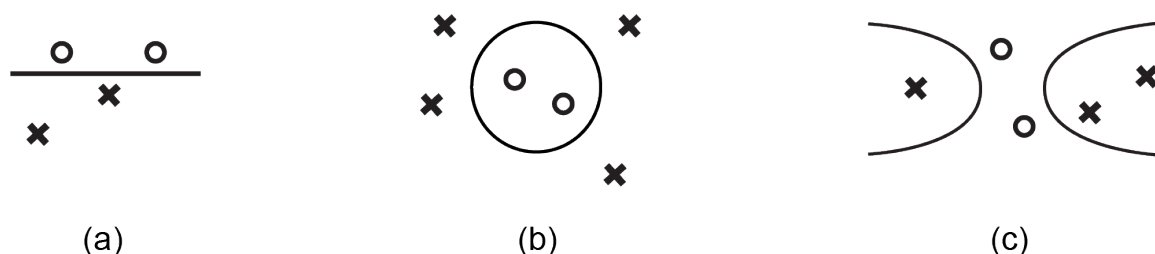
5.2 Redes de função de base radial

Uma rede de base radial (*Radial Basis Function* – *RBF*) difere de um *perceptron* multicamada por conter apenas três camadas: uma de entrada de dados, uma camada oculta de neurônios e uma camada de saída. A camada oculta contém um tipo especial de neurônios que fazem a transformação não linear do sinal de entrada para o espaço dos neurônios ocultos. Esse espaço oculto tende a ter uma alta dimensionalidade. Um problema de classificação tem maior probabilidade de ser linearmente separável quanto maior a for dimensionalidade (denominado de Teorema de Cover; isto pode ser questionado a partir da maldição da dimensionalidade que será vista adiante). As RBFs podem ser utilizadas também para reconhecimento e classificação de padrões e regressão linear. A camada oculta se conecta com a camada de saída por meio de pesos sinápticos.

Os neurônios ocultos transformam os dados de entrada mediante o uso de *funções de base radial*. Uma função de base radial tem a característica de possuir um parâmetro identificado como o centro da função de base radial, a partir do qual os valores de entrada estão mais distantes ou mais próximos do valor do centro (ou seja, podem ter um *raio* maior ou menor). Geralmente são utilizadas funções como a função *gaussiana*, ou ainda as da classe das *quádricas* (Haykin, 2001, p. 292). Essas funções permitem obter uma separabilidade além da linear que é proporcionada pelo *perceptron*, podendo ser *esférica* (no caso de gaussianas) ou *quádrica*, ou seja, curvas generalizadas por seções cônicas – parábolas, hipérboles ou elipses (Figura 21). As RBFs possuem, além dos pesos sinápticos, estes parâmetros livres que são o centro da função de base radial e o espalhamento em relação a este centro (variância, no caso de uma função

gaussiana). Esses parâmetros também podem ser ajustados em um algoritmo de treinamento da RBF.

Figura 21 – Exemplos de (a) separabilidade linear; (b) separabilidade esférica; (c) separabilidade quádrlica



As RBFs podem ser utilizadas de forma eficaz em problemas de classificação em espaços não lineares, em problemas de interpolação de dados e em redes de regularização, tornando a RBF como um *aproximador universal de funções*. Uma RBF difere de uma MLP por ter, portanto, uma única camada oculta; são não lineares (possuem funções de base radial), a função de ativação calcula uma norma euclidiana em relação aos centros das funções de base radial, em vez da MLP que utiliza funções que simulam um disparo; e as RBFs permitem a construção de aproximações locais no mapeamento de entrada e saída, em vez da MLP que busca uma aproximação global (Haykin, 2001, p. 323).

5.3 Máquinas de vetor de suporte

Enquanto das redes MLP e RBF se baseiam na obtenção de um mapeamento de entrada e saída, uma máquina de vetor de suporte (*Support Vector Machine – SVM*) se concentra no cálculo e obtenção de um hiperplano ótimo, que minimize da melhor forma a separação entre as classes dos padrões (Haykin, 2001, p. 350). Os vetores de suporte são os dados de treinamento utilizados no cálculo que se posicionam nos limites das classes e que servem de base para o traçado dos hiperplanos. O processo de treinamento é feito por meio de um processo de maximize a separação entre os padrões.

Para as SVM, é necessário ter um conjunto de padrões pré-classificados, de maneira a consistir em uma RNA com aprendizagem supervisionada. Existe uma função custo que orienta o processo de treinamento e que indica o nível de separabilidade que é alcançado pela SVM. Para o algoritmo de treinamento, pode

ser utilizado o método dos multiplicadores de Lagrange, que minimiza a função custo na forma de uma função lagrangiana (Haykin, 2001, p. 354).

As SVMs são atrativas no sentido de garantirem a obtenção de um extremo global da função custo, gerando um conjunto de hiperplanos que irão separar adequadamente o conjunto de entrada, e podem ser implementadas de forma eficiente, permitindo lidar com um conjunto de dados com dimensão bastante elevada. Entretanto, a tendência é que elas tenham um treinamento mais lento que os outros tipos de RNA, por não haver um controle sobre quais dados podem ser utilizados como vetores de suporte; também não há como embutir conhecimento prévio na rede. Ainda assim, as SVMs são alternativas bem interessantes para tarefas, como a regressão não linear (Haykin, 2001, p. 378).

5.4 Análise de componentes principais

No reconhecimento de padrões, uma das etapas necessárias à tarefa é a extração de características (Figura 15). Se um espaço de dados utilizados para a entrada de uma RNA possui uma cardinalidade alta, é oportuno supor que exista algum grau de redundância nestes dados. Desta forma, a extração de características iria priorizar os dados que realmente conseguem distinguir adequadamente os padrões de entrada de maneira a diminuir este espaço de características. Se cada característica ocupa uma entrada da RNA, que também significa uma dimensão deste espaço de entrada, a extração de características pode então efetuar uma *redução de dimensionalidade* (Haykin, 2001, p. 433).

Esse processo de reduzir a dimensionalidade está ligado ao que se denomina de *maldição da dimensionalidade* no estudo de RNA. A maldição da dimensionalidade está ligada ao fato de que espaços de dados com um número muito grande de dimensões podem aumentar exponencialmente o nível de complexidade no processo de classificação. Ou seja, seriam necessários mais dados de treinamento à medida que aumenta o número de dimensões/entradas para que a RNA pudesse classificar corretamente tal conjunto de dados.

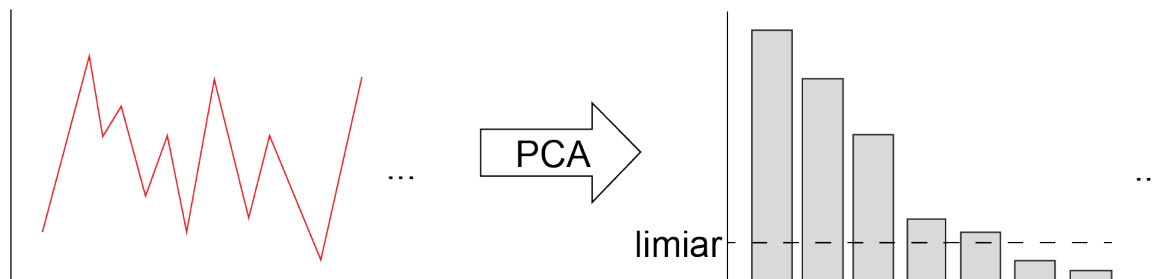
Uma função definida em um espaço de alta dimensionalidade é provavelmente muito mais complexa do que uma função definida em um espaço de baixa complexidade, e estas complicações são mais difíceis de se perceber. (Friedmann, 1995, citado por Haykin, 2001, p. 238)

Nascida no âmbito do tratamento estatístico de dados (Medeiros; Garcia, 2019; Hair Jr. *et al.*, 2009), a análise de componentes principais (*Principal*

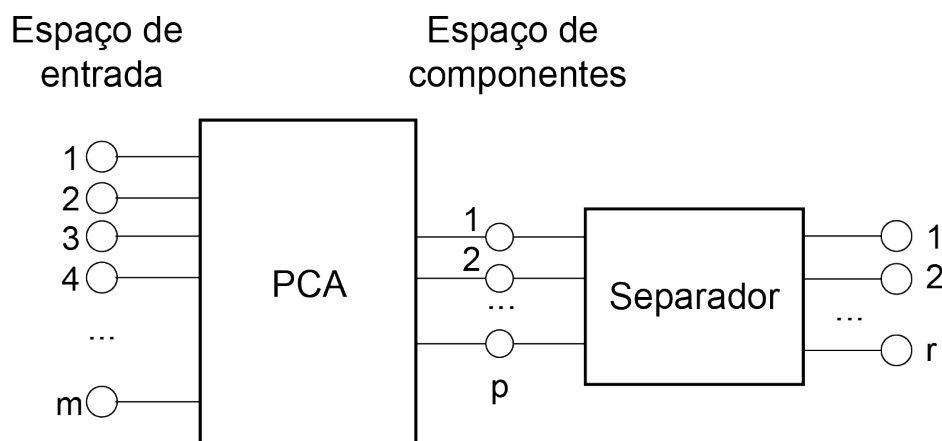
Component Analysis – PCA) fornece uma técnica efetiva para a redução de dimensionalidade (Haykin, 2001, p. 438). A PCA é uma transformação de dados que substitui um espaço de entrada que pode estar correlacionado, ou seja, a variância numa dimensão/entrada pode ser influenciada pela variância de outra dimensão/entrada. A PCA substitui esse espaço de entrada por um conjunto de variáveis (componentes) que estão descorrelacionados entre si, ordenando pelos que possuem maior variância para os de menor variância. Esses componentes (na forma de autovalores) são então utilizados como entradas para a camada do separador propriamente dito de uma RNA, podendo haver um limiar de corte: a partir de determinado valor, as componentes variam tão pouco que são inócuas para o treinamento do separador.

Na Figura 22 pode ser visto esse processo. A obtenção dos componentes descorrelacionados (ou, colocado de outra forma, devidamente ortogonalizados), transforma um espaço m -dimensional dos padrões de entrada em um espaço p -dimensional de componentes, com $p < m$, faz com que o número de pesos sinápticos seja menor, proporcionando mais eficiência para a aprendizagem no estágio do separador.

Figura 22 – Em (a), a PCA faz a transformação dos dados de entrada em um novo conjunto de componentes descorrelacionados entre si. Em (b), um espaço de entrada de m dimensões é substituído por um espaço de p componentes que irão classificar os padrões em r classes



(a)



(b)

5.5 Mapas auto-organizáveis

Mapas auto-organizáveis (*Self Organizing Maps – SOM*) são RNA onde os neurônios estão dispostos em um reticulado (grade retangular ou hexagonal) que possui geralmente uma ou duas dimensões. Utilizando a aprendizagem competitiva, os neurônios são localizados dentro desse espaço uni ou bidimensional, de acordo com a proximidade dos padrões de entrada (Kohonen, 1998). A aprendizagem é não supervisionada, e no caso das SOM é criado uma representação topográfica dos dados.

Um mapa auto-organizável é, portanto, caracterizado pela formação de um mapa topográfico dos padrões de entrada no qual as localizações espaciais (i.e., coordenadas) dos neurônios na grade são indicativas das características estatísticas intrínsecas contidas nos padrões de entrada, daí o nome “mapa auto-organizável”. (Haykin, 2001, p. 483)

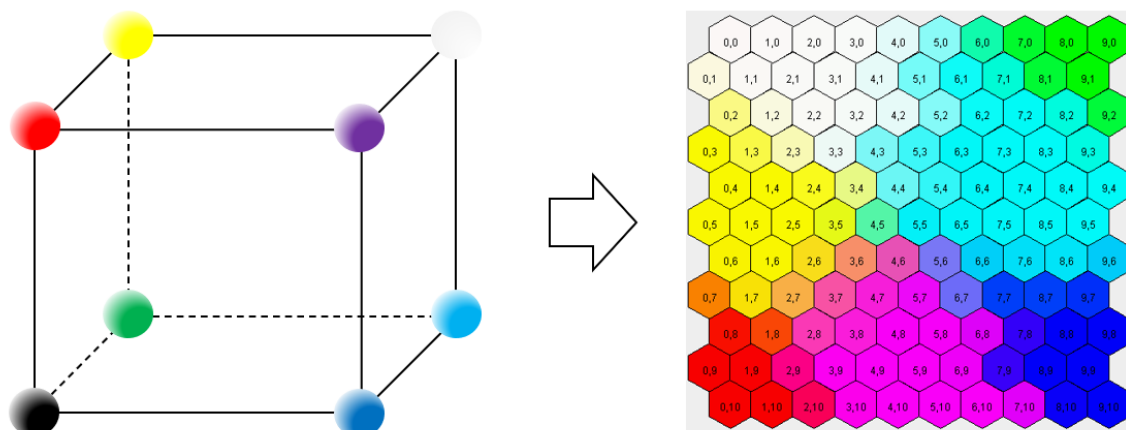
Diz-se que SOM perfaz um *ordenamento topológico* dos dados. O algoritmo responsável pela formação do SOM inicia com os pesos sinápticos aleatórios. Após, existem três processos essenciais para a formação do mapa topográfico (Haykin, 2001, p. 487), como se seguem.

1. **Competição:** para cada padrão de entrada, os neurônios calculam suas funções discriminantes (geralmente a função euclidiana ou outra função de distância). Aquele neurônio que possui o maior valor discriminante (ou o menor, fazendo referência a uma função distância) será considerado o neurônio vencedor.
2. **Cooperação:** o neurônio vencedor determina a região de vizinhança topológica, no qual os vizinhos são excitados, acontecendo a cooperação entre os neurônios vizinhos a um neurônio vencedor. A região de cooperação tende a ser grande no início do algoritmo de treinamento e vai decaindo ao longo das iterações.
3. **Adaptação sináptica:** permite que os pesos sinápticos relacionados com os neurônios excitados melhorem suas funções discriminantes (ou seja, são aproximados ao neurônio vencedor) em relação aos padrões de entrada, acontecendo de forma gradativa.

Após o treinamento, além da característica de ordenamento topológico, existe o *casamento de densidade*: o SOM tende a refletir nas distribuições dos neurônios a densidade dos dados de entrada, podendo haver concentrações de neurônios ou espaços mais rarefeitos no mapa topográfico. Uma rede SOM também permite a *seleção de características*: de um conjunto de dados de entrada não linear, SOM é capaz de proporcionar uma distinção das características que são melhores para se fazer a aproximação a um conjunto de classes. Por causa disso, uma rede SOM pode ser utilizada também em reconhecimento de padrões na fase de extração de características.

Um exemplo didático para mostrar o potencial de SOM é o mapeamento ou projeção do sistema de cores RGB em três dimensões (com cada cor principal em uma aresta oposta no cubo) para um mapa auto-organizável bidimensional. O mapa condiciona topologicamente as cores por sua similaridade.

Figura 23 – Exemplo de SOM para a projeção do espaço de cores RGB em três dimensões (esquerda) para um mapa auto-organizável em duas dimensões, com uso de reticulado hexagonal (direita)



REFERÊNCIAS

HAIR JR., J. F. *et al.* **Análise multivariada de dados**. Porto Alegre: Bookman, 2009.

HAYKIN, S. **Redes neurais**: princípios e prática. Porto Alegre: Bookman, 2001.

HEBB, D. O. **The organization of behavior**. New York: John Wiley & Sons, Inc, 1949.

KOHONEN, T. The self-organizing map. **Neurocomputing**, v. 21, n. 1-3, p. 1-6, 1998.

MEDEIROS, L. F. de. **Inteligência artificial aplicada**: uma abordagem introdutória. Curitiba: InterSaberes, 2018.

_____. **Redes neurais em Delphi**. 2. ed. Florianópolis: Visualbooks, 2007.

MEDEIROS, L.; SILVA, H.; RIBEIRO, E. Tomographic Image Reconstruction of Fan-Beam Projections with Equidistant Detectors using Partially Connected Neural Networks. **Learning and Nonlinear Models**, v. 1, n. 2, p. 122-130, 2003.

MEDEIROS, L. F.; GARCIA, M. S. S. Principal component analysis on the students' perception of a cognitive assistant for content reinforcement in Higher Education. In: LYTRAS, M. D. et al. (org.). **Cognitive computing in technology-enhanced learning**. [S. I.]: IGI Global, 2019. p. 63-87.

NIELSEN, M. **Neural networks and deep learning**. [S. I.]: Determination Press, 2018.